VDK/TLM Creation

Rapid Training

X-2025.06

Module 1: Lab D

Model Debugging

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview *2*

Task-1: Open the Timer Project **2**

Task-2: Implementing the Interrupt Clearing **3**

Task-3: Debugging your Model **4**

Linux Setup 4

Windows Setup 4

Task 3 a) Built-in Debugger of Virtualizer Studio (Linux) 5

Task-4: What's Next **7**

Lab Overview

Your task for this lab is to implement one additional aspect of the timer behavior, debug the code and fix the issues. For debugging we will use multiple techniques, starting from using built-in FastTrack messages in callbacks and reuse objects, using a source code debugger to step through and analyze the code, as well as using VDK Debug features to inspect the value of registers on your model.

Task-1: Open the Timer Project

***Motivation**: We have already created basic version of a programmable interval timer model, including a basic testbench in Lab A through Lab C.*

If you have completed the previous lab (Module 1 Lab C), including the optional task of creating a unit test for the interrupt flag clearing behavior, you can open Virtualizer Studio and continue with the same timer.tlmc project and skip this task. Continue with Task-2.

1. If have not completed the previous lab or want to start from a prepared starting point for this lab, use the following steps to create the starting point.

   i. Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

   ```
   % source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)

   % source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)
   ```

   This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

   i. Go to the `<Lab-data>/data/module_1` directory and start Virtualizer Studio.

   ```
   vs -d start_Module1LabD &
   ```

2. Close the welcome screen and import the starting point TLM Creator Project by clicking ***File*** → ***Import…*** menu option.

   i. In the ***Import*** dialog, select ***General*** → ***Existing Projects into Workspace*** to select what you want to import.

   ii. Click ***Next*** to get to the ***Import Projects*** dialog. Browse to the `timer` directory in the `start_Module1LabD` directory and click ***Finish*** (Make sure you are in the right directory – it's easy to end up in the wrong directory!).

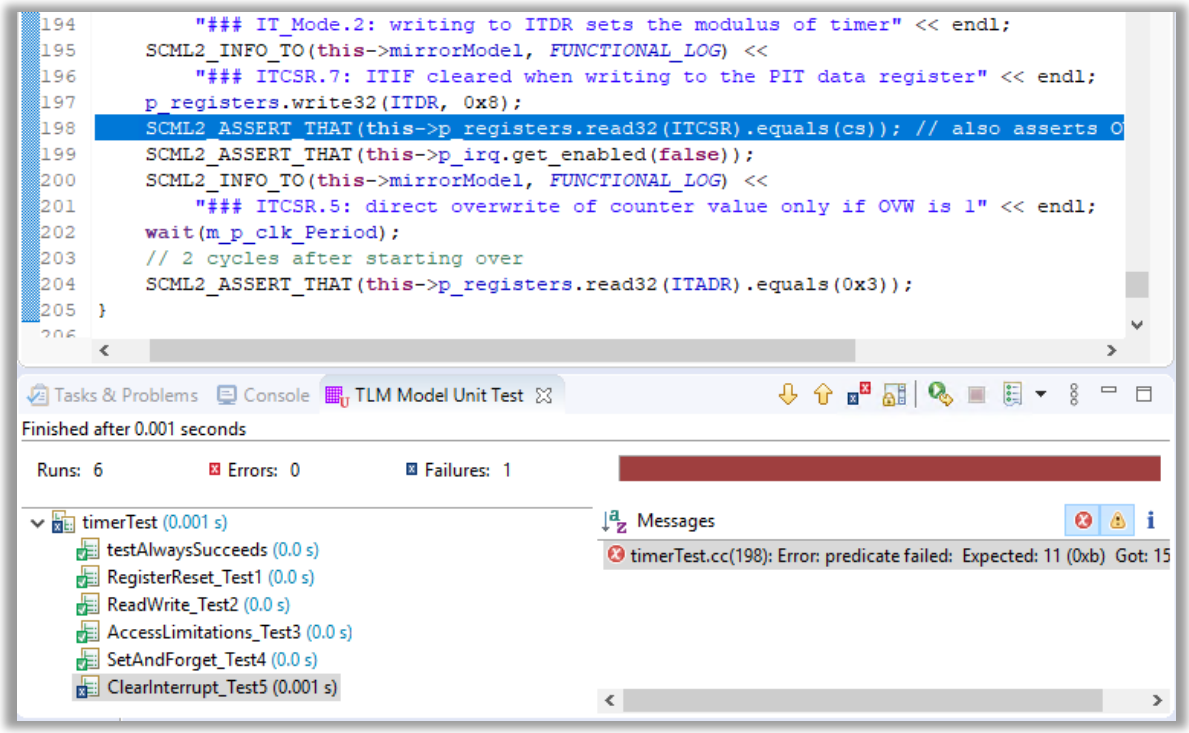
   iii. Select the `timer.tlmc` file in the Project Explorer and click ***Generate Sources Code*** from the ***Build*** toolbar button options. Also, generate the documentation using the ***Generate Documentation*** option.

This will provide you with the starting point `timer` model for this lab.

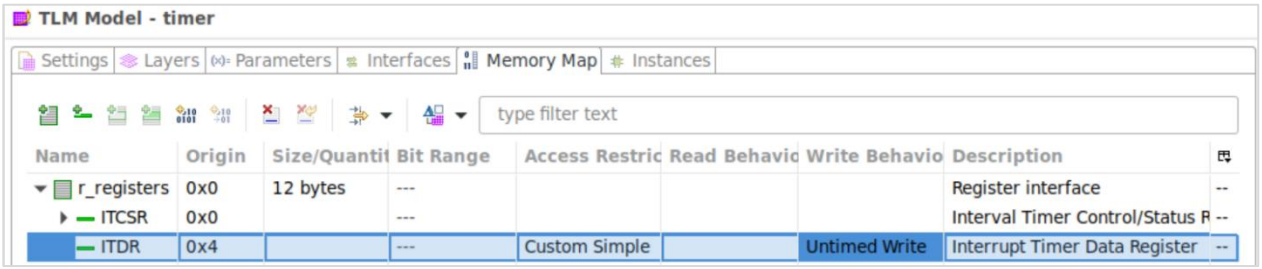
Task-2: Implementing the Interrupt Clearing

Motivation: *In the last lab (Module 1 Lab C), we described a test setup for clearing the interrupt of the timer by writing to its data register ITDR – if you have implemented that test, you can use your test, or otherwise the test* `ClearInterrupt_Test5` *that is provided with the starting point for this lab.*
We will first run this test to see where it fails and then implement the missing functionality. Finally, we will run the test again to verify the implementation passes the test.

3. Run the unit tests for the timer model by clicking the ***Test Run*** ( ) button from the toolbar. As expected the unit test for the interrupt clearing behavior fails – after writing to the *ITDR* register the interrupt flag should have been cleared and the interrupt signal de-asserted.



To implement this behavior, you need to clear the interrupt flag after the CPU has written to the *ITDR* register. For that, go to the ***Memory Map*** page of the Model editor and add an *Untimed Write behavior in the* ***Write Behavior*** *column,* for the *ITDR* register.



4. Save the model to trigger the generation of the default implementation of the appropriate callback. Replace the `// FIXME` comment in the ***Behavior*** subtab, by a line setting the *ITIF* flag to `true`:

```
bool timer::handle_write_ITDR(const unsigned int& value,
             const unsigned int& byteEnables) {
      // clear interrupt flag when CPU writes ITDR register
      this->ITCSR.ITIF = true;
      this->ITDR = ((this->ITDR & ~byteEnables) | (value & byteEnables));
      return true;
}
```

5. Rebuild the model and run the test again – unfortunately, the test still fails at the same line, so something did not work out as planned. In the next section you will debug this.

Task-3: Debugging your Model

Motivation: *Occasionally, debugging at the platform level points to a problem with one of the underlying transaction level models of the virtual prototype. When this happens, you want to easily switch from platform-level debugging down into a powerful source code debugger.*

Typically, you use source-code debugging in combination with the SystemC-aware debugging provided in Virtualizer Studio, e.g. by also viewing signal and register values of the VDK Debug perspective.

We will be describing the source code debugger flow for both Linux and Windows separately.

Linux Setup

No special installation or setup steps are needed for using the built-in source code debugging of Virtualizer Studio.

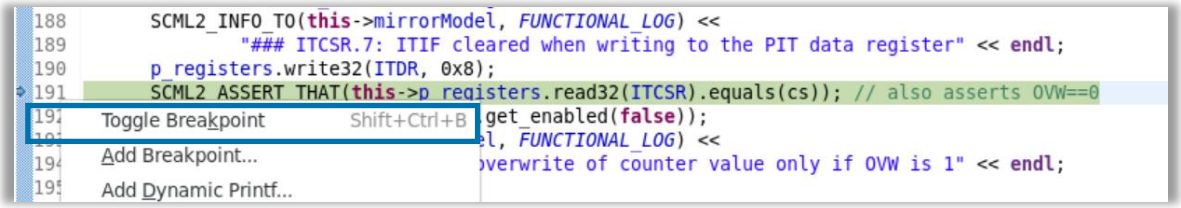
Windows Setup

Note: The Windows Source Code Debugger flow has some issues and may not work as expected. Refer to the *VP Support* for more details. In this Lab exercise, we will cover the debugging steps for the Linux environment only.

Task 3 a) Built-in Debugger of Virtualizer Studio (Linux)

Note: Windows users can skip this task.

6. First, set a breakpoint for the line with the failing assertion in the `timerTest.cc` file, by right-clicking on the left margin and selecting ***Toggle Breakpoint*** option from the context menu, or by using the keyboard shortcut `<Shift+Ctrl+B>`.



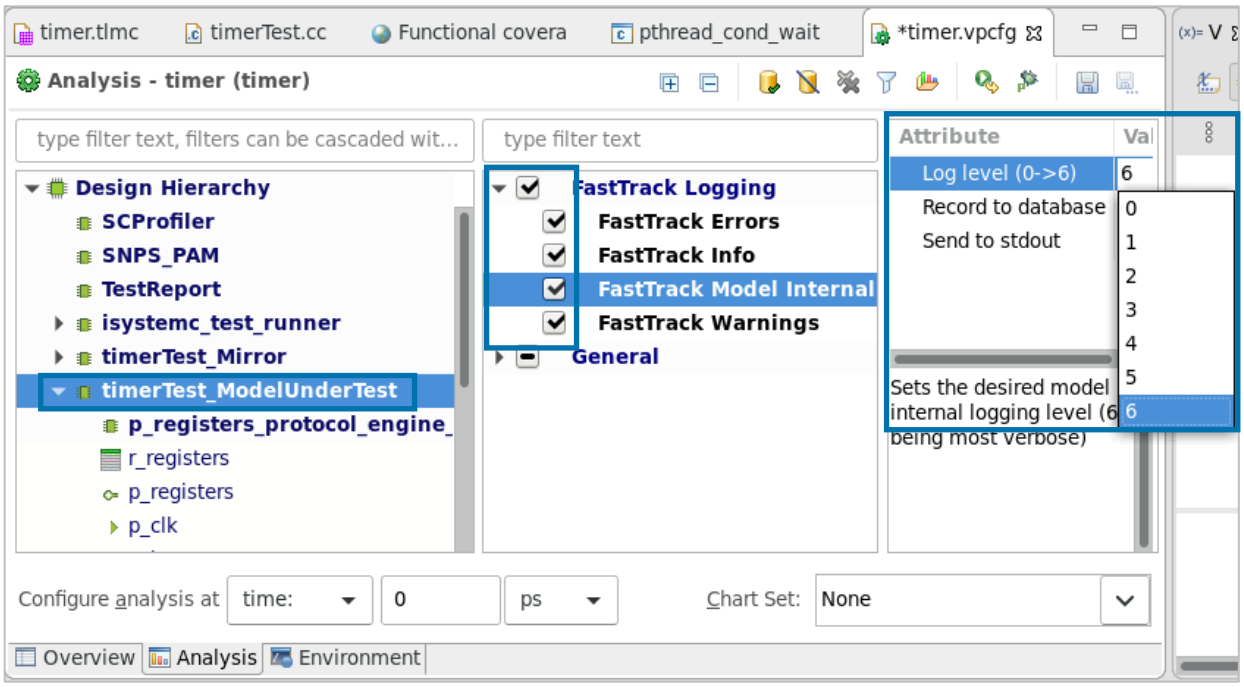
Launch the unit tests with the debugger attached: Click on the ***Debug*** button ( ) on the toolbar and select ***Debug As > TLM Model Unit Test*** option. This opens the Debug perspective of Virtualizer Studio.

7. Click ***Build using Debug Build Settings*** to proceed with the Debug settings.

The debugger stops first at the beginning of the simulation (time 0 at `initial_crunch` breakpoint).

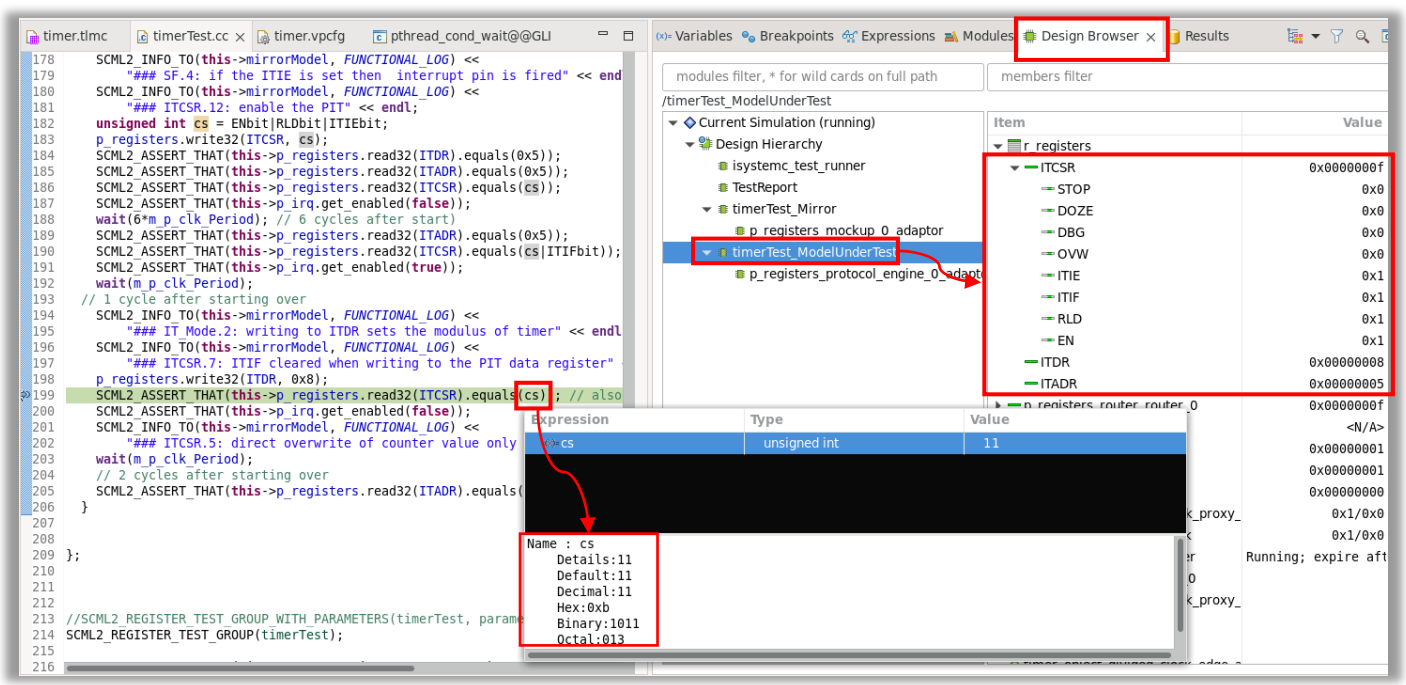
8. Before we continue, enable all available FastTrack messages, to have the models, including the reuse objects, spit out more detailed information: click on the ***Configure Analysis Settings*** button in the right-hand (VDK Debug) side of the top level menu to open the ***Analysis*** dialog of the *timer VP Config*.

9. In the analysis settings, make sure that all FastTrack Logging items are enabled. Also configure the FastTrack Model Internal monitor to log all categories, from level `0` to `6`:



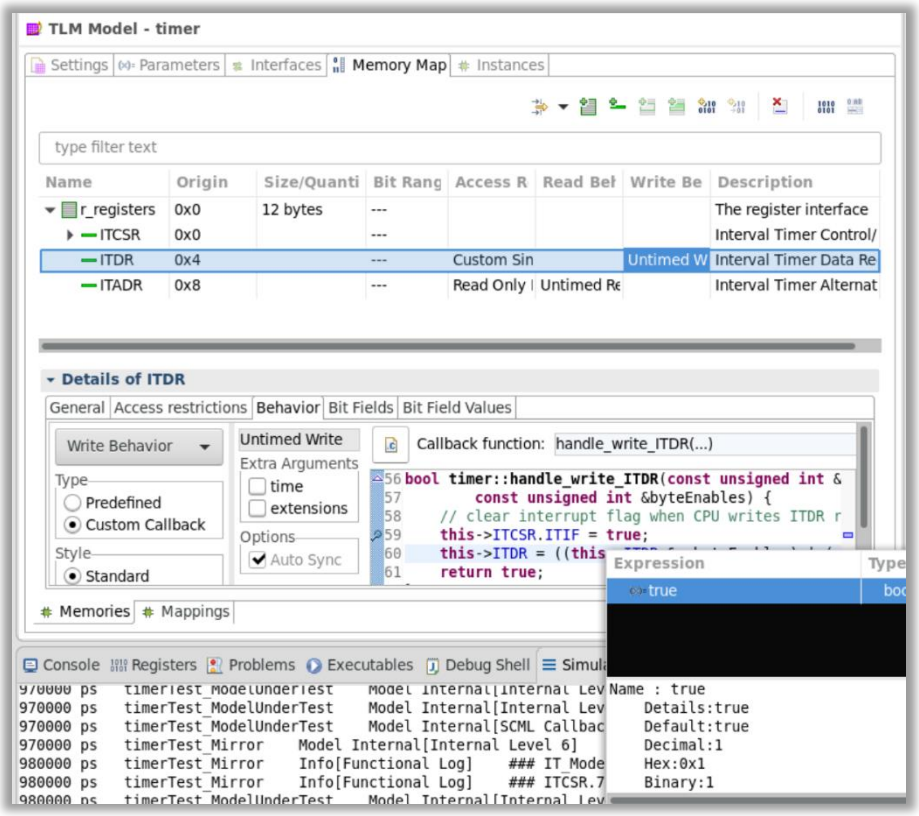
10. To continue running until we hit our breakpoint, press the ***Resume*** button in the top-level menu or hit `<F8>`. This will continue the simulation until our breakpoint in the `timerTest.cc` file is hit.

    The assertion fails because the state of the ITCSR register is not equal to cs – you can see the value of cs by hovering your mouse over the cs value in the line, and the value of the ITCSR register, including the meaning of the different bits, you can best see by selecting the Design Browser tab, selecting the model under test, and then the ITCSR register:



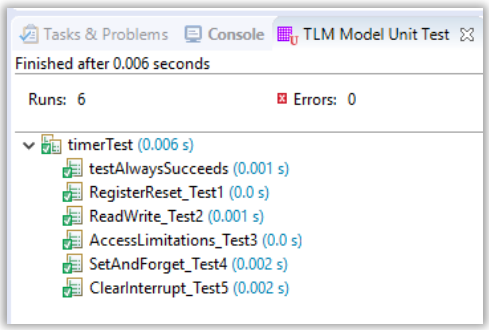
From the debug views above you can see that the interrupt flag *ITIF* is still set, though it should not. The next place to check would be hence the `handle_write_ITDR` callback (either in the timer.cc file or in the Memory Map's *ITDR* ***Behavior*** section.
Hovering your curser over the `true` constant assigned to `ITCSR.ITIF` is translates to `0x1`, but what you want/need here is `0x0`. Therefore, change the line to

```
this->ITCSR.ITIF=0x0;
```



11. Stop the currently running simulation and debugger using the ***Terminate*** button ( ) or pressing <Ctrl+F2>, switch back to the TLM Creation perspective ( ) of Virtualizer Studio, save the project, and re-build the timer model.

12. Next, run all the unit tests again by pressing the ***Test Run*** ( )button in the top-level menu. Now you should see all 7 tests passing:



Task-4: What's Next

In the next lab, we will integrate the Timer model into a VDK project using the VDK Creation perspective of Virtualizer Studio.